

Debugging ML code

ONE LOVE. ONE FUTURE.

1

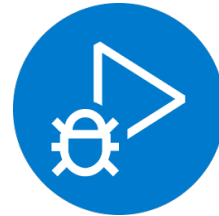
Debugging is a hard but necessary discipline



2

Debugging ML code is even harder

- Bugs in ML code can be non-ml specific and ML specific
- Classic bugs: Code does not run
 - Use traditional debugger to find these
- ML specific bugs: My model is not converging
 - Need the correct approach for debugging



Lets start with the classics...



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

3

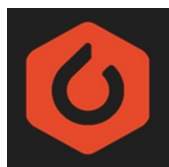
First step: Get a good environment

💡 Jupyter notebooks are great at what they are meant for: exploring ideas and combining code + text into standalone document...

💡 However, it can be a pain debugging code in notebooks...



VS Code



Torchstudio



Pycharm



SPYDER

Spyder



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

4

Python debugging in general

1. Print statements
2. Stop at an interesting point in your code and interact
3. Work in an actual debugger

```
print("x.shape = {}".format(x.shape))
```

```
from IPython import embed; embed()
... # do your stuff interactively here
exit() # exit ipython to let your code continue
```

```
import pdb; pdb.set_trace()
```

Learn more here

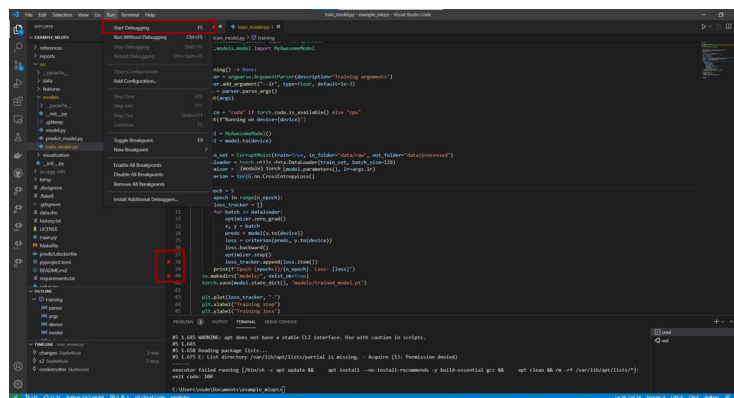
<https://switowski.com/blog/ipython-debugging>



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

5

VS code debugger



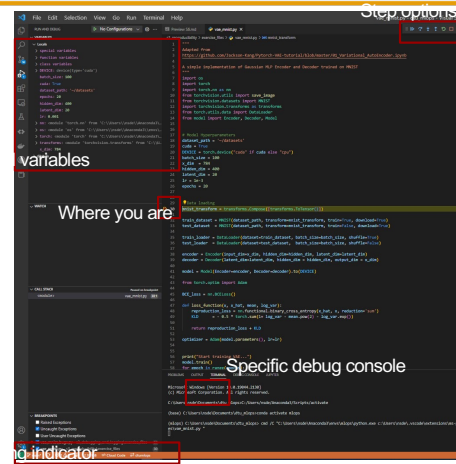
ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

6

VS code debugger

Step options:

- F5: next breakpoint
- F10: next line
- F11: step into
- Shift-F11: step out
- CTRL-Shift-F11: Restart
- Shift-F5: stop



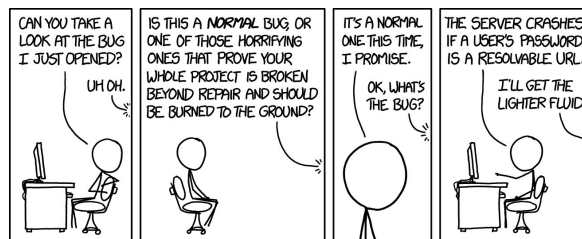
ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

7

Back to these ML specific bugs

Or better know as

⚠ When everything is running, but results are wrong



Finding these buggers comes with experience

A potpourri of my findings over the last couple of years and [others](#).



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

8

Steps in ML debugging

1. Check your data
2. Start as simple as you can
3. Make everything deterministic
4. Investigate the math
5. Overfit to your data
6. Look at your loss
7. Visualize everything
8. Add complexity in steps



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

9

1. Check your data!

Your starting point should always be the data in ML!

Check

- ✓ Examine summary statistics (data normalized?)
- ✓ Look at label distributions (is it shuffled?)
- ✓ Visualize a few samples (are they corrupted in anyway?)

If you are working on datasets you know, you may skip these steps.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

10

2. Start as simple as you can

Remove all the fancy stuff

- Mixed precision
- Regularization like dropout
- Early stopping
- Learning rate schedulers
- ...

One or more of these may be enabled by default if you are starting from someone's else codebase



Prompt:
Machine Learning bot with a fancy hat



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

11

3. Make everything deterministic

Every machine learning run is by default random. Try fixing:

- 💡 Seed everything and use same seed everywhere
- 💡 Remove all data augmentation
- 💡 Use only a single batch of data where you have a feeling of the outcome

```
int getRandomNumber()
{
    return 4; // chosen by fair dice roll.
             // guaranteed to be random.
}
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

12

4. Investigate the math

Debug the math:

- 💡 Go through your code, line by line
- 💡 There should be a one-to-one match between equations and lines of code
- 💡 Refactor if it is not clear

Check dimensions and annotate if necessary or use [typing software](#)

```
# add shape comments
A = torch.randn(N, D) # Nx D
x = torch.randn(D) # D
Ax = A.mv(x) # N
```

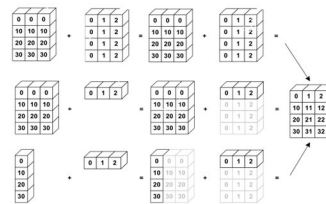


ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

13

4. Investigate the math (continue)

- 💡 Lookout for broadcasting!
- 💡 Broadcasting in python is both a blessing and a curse
- 💡 It can create problems (real life example)



```
import torch
preds = torch.randn(100,)
target = torch.randn(100,1)
loss = (preds - target).abs().pow(2.0).sum()
```

What is the problem here?



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

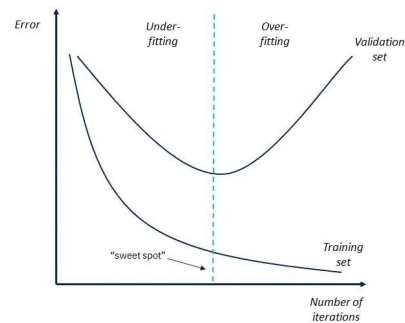
14

5. Overfitting is good?

Overfitting is usually seen as an bad thing bug...

💡 Models should be able to memorize one batch

💡 Train on one batch, if loss is 0 move on to larger models/large data else debug



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

15

6. Really look at your loss

- Assuming your model is training without errors, but your loss is not behaving as it should.
 - 💡 Are you printing/logging the results correctly?
 - 💡 Did you remember `loss.backward`, `optimizer.step` and `optimizer.zero_grad`?
 - 💡 What about your learning rate and batch size?
- For your loss, if possible, calculate in log-space

```
a = 1
for x in data:
    a = a * x

log_a = 0
for x in data:
    log_a = log_a + log(x)
```



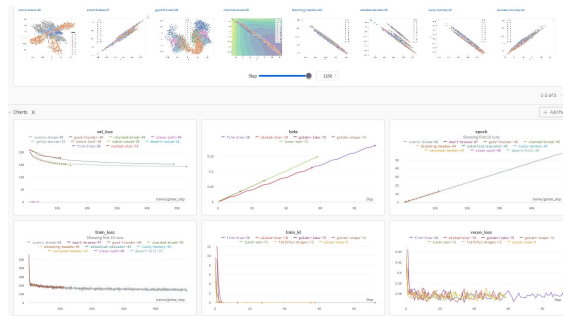
ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

16

7. Visualizations are your friend

Log and visualize everything

- 💡 Training loss is really decreasing right?
- 💡 Can you add additional metrics?
- 💡 Log dynamic changing hyperparameters (learning rate with lr schedulers)
- 💡 Plot data, predictions, reconstructions etc. over time



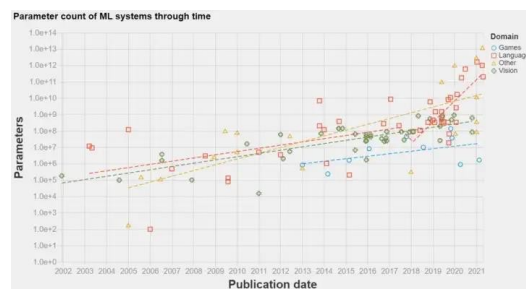
ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

17

8. Add complexity over time

If you are still good, then

- Get a baseline that just works (=train on more data data)
- Stop overfitting:
 - Add back regularization
 - Add back data augmentations
- Tune hyper parameters



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

18

Debugging

- Debugging is a skill that improves with experience
- Common but inefficient debugging method: `print(...)` statements
- Effective debugging needed for large codebases
- **Solution:** Familiarize with Python's built-in debugger, `pdb`

Invoking Python Debugger

- Set trace with `pdb` by inserting:

```
import pdb
pdb.set_trace()
```
- Use commands to step through the code (see cheatsheet)

Debugging in Editors

- In VS Code, use inline breakpoints (F9) to stop the execution
- Execute the script in debug mode
- Inline breakpoints = small red dots to the left of code



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

21

Debugging with Errors

- Start the debugger automatically when errors occur:
`python -m pdb -c continue my_script.py`



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

22

Profilers Overview

- Profiling focuses on improving code performance
- In this session: focus on **runtime**
- Two key questions:
 - How many times is each method called?
 - How long does each method take?
- Profilers help identify bottlenecks in code



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

25

cProfile Overview

- Built-in Python profiler
- Provides an overview of function and method runtime
- Usage:

```
python -m cProfile -o <output_file> -s <sort_order>  
myscript.py
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

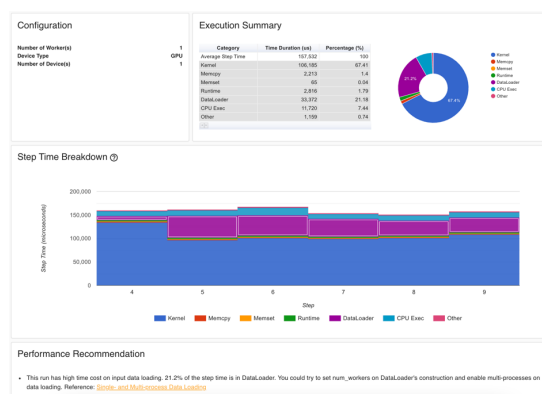
26

Exercises for cProfile

1. Analyze profiling output: Which function takes the longest to run?
2. Difference between `totttime` and `cumtime`
3. Visualize profiling with `snakeviz`
4. Optimize the run and verify speedup

PyTorch Profiling

- Profiling ML code involves mixed devices (CPU+GPU)
- Aim: Identify bottlenecks in overlapping computations
- PyTorch has its own profiler for complex applications



PyTorch Profiler Example

- Example:

```
with torch.profiler.profile(...) as prof:  
    output = model(data)
```

- Tracks kernel time and transfer times (e.g., memcopy)
- Provides recommendations



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

29

Exercises for PyTorch Profiler

1. Explore the [API for the profiler](https://pytorch.org/docs/stable/profiler.html)
2. Run a profiling example using ResNet18
3. Identify operations taking most CPU time
4. (Optional) Profile with GPU

<https://pytorch.org/docs/stable/profiler.html>



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

30

Example

```
import torch
import torchvision.models as models from torch.profiler
import profile, ProfilerActivity
model = models.resnet18()
inputs = torch.randn(5, 3, 224, 224)
with profile(activities=[ProfilerActivity.CPU],
             record_shapes=True) as prof:
    model(inputs)
```

Optimizing with PyTorch Profiler

- Profile memory usage with additional arguments
- Export profiling results with:
`prof.export_chrome_trace("trace.json")`
- Visualize with Chrome tracing (`chrome://tracing`)

Advanced PyTorch Profiling

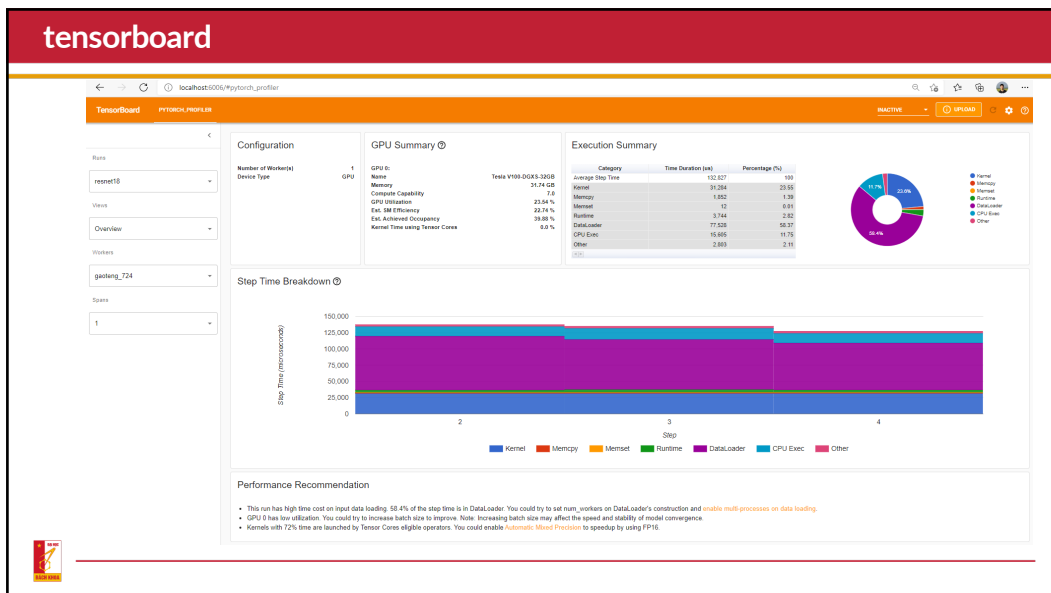
- Profile multiple iterations to avoid single-sample bias:

```
with profile(...) as prof:
    for i in range(10):
        model(inputs)
        prof.step()
```

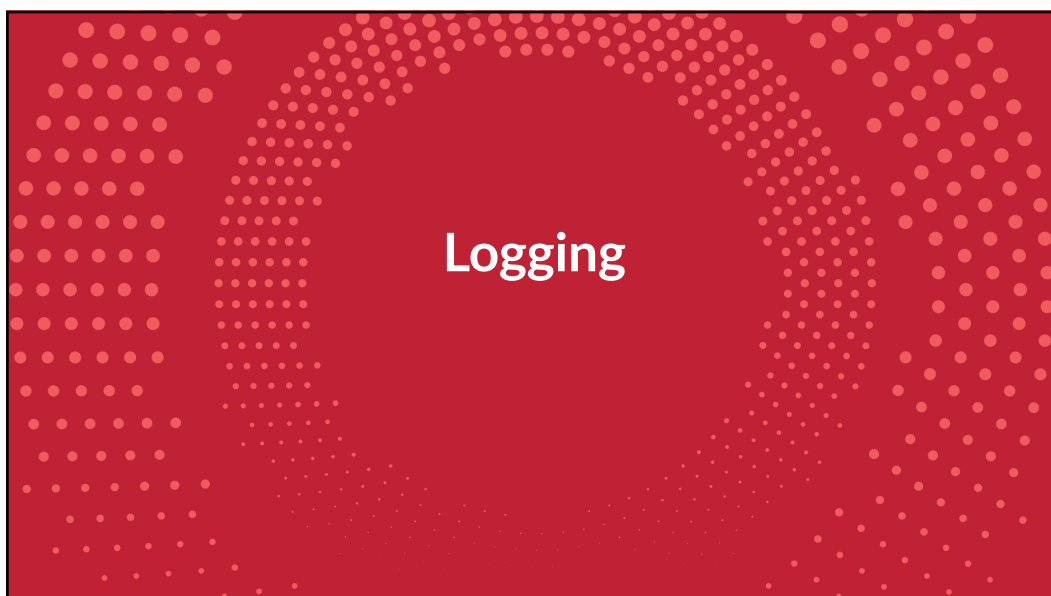
Visualizing Results with TensorBoard

```
from torch.profiler
import profile, tensorboard_trace_handler
with profile(...,
on_trace_ready=tensorboard_trace_handler("./log/resnet18")) as prof:
    ...
```

- Use TensorBoard to visualize profiling:
tensorboard --logdir=./log
- Explore the interface and compare runs using the diff tab



35



36

What is Logging?

- Core module in application development.
- Records events and activities over time.
- Benefits of proper logging:
 - Easier debugging.
 - Monitoring app health and performance.
 - Auditing activities for future reference.
 - Analyzing app behavior for insights.

Types of Logging

- **Application logging:** Useful for any type of application.
- **Experiment logging:** Essential for machine learning projects involving experiments.

Application Logging

- Basic example: Python print statement.

```
for batch_idx, batch in enumerate(dataloader):  
    print(f"Processing batch {batch_idx} out of  
          {len(dataloader)}")
```

- Useful but limited; Python's logging module provides more flexible logging.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

39

Python Logging Components

1. **Logger:** Entry point to the logging system.
2. **Handler:** Directs log messages (e.g., console, file).
3. **Formatter:** Specifies log layout (e.g., timestamps).
4. **Level:** Sets severity of logs.



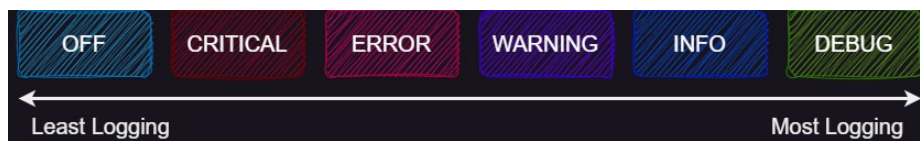
ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

40

Logging Levels

- Use levels to filter log outputs:

```
logger.debug("Used for debugging.")
logger.info("Informative messages.")
logger.warning("There is a warning.")
logger.error("An error occurred.")
logger.critical("Critical error.")
```



Logging vs Error Handling

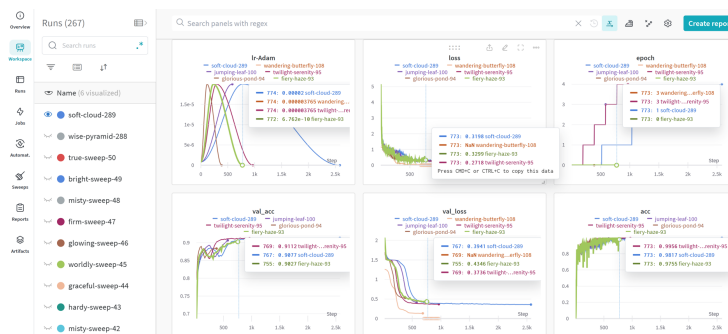
- Logging provides information after a program runs.
- Error handling (try, raise) adjusts program flow at runtime.

```
def f(x: int):
    if not isinstance(x, int):
        raise ValueError("Expected an integer")
    return 2 * x
```

Experiment Logging

- Essential for tracking model performance during training.
- Key tools:
 - [Tensorboard](#)
 - [Weights & Biases](#)
 - [MLFlow](#)
- Weights and Biases (wandb) used in this course for logging experiments.

Weights and Bias (wandb) dashboard



Exercises: Setting up Logging

1. Create a basic logger:

```
import logging
logging.basicConfig(level=logging.DEBUG)
logger = logging.getLogger(__name__)
```

2. Use logging configuration to output to terminal and files.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

45

Experiment with Weights & Biases

1. Create a wandb account.
2. Install wandb: `pip install wandb`
3. Now connect to your wandb account: `wandb login`
4. Log experiments by integrating wandb into your code:

```
wandb.init(project="example")
wandb.log({"loss": loss_value})
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

46